

DIT Parking Manager - Simple Project Explanation

Project: Parking management system for Dar es Salaam Institute of Technology (DIT).

Main idea: Hii application inasaidia kusimamia parking: magari, parking slots, check-in, check-out, payment, tickets, na reports.

Stack: Frontend: React + TypeScript + Vite. Backend: Spring Boot + Java. Database: PostgreSQL.

1. Project Logic Kwa Urahihi

1. User ana-login kama **Admin** au **Customer**.
2. Customer ana-register gari lake, mfano car, truck, au motorcycle.
3. System inaonyesha parking slots zilizopo: Standard, VIP, na Disabled.
4. Vehicle ikiingia parking, system inafanya **check-in**, inachagua slot, na kutoa ticket.
5. Vehicle ikitoka, system inafanya **check-out** na kuhesabu fee.
6. Customer/admin analipa kwa Cash, Card, au Mobile Money.
7. Admin anaweza kuona reports za revenue na occupancy.

2. Admin Anafanya Nini?

Kwa real life, Admin ni kama parking manager au parking attendant. Yeye ana access kubwa ya kusimamia parking nzima.

Admin Role	Maelezo Ya Real Life	Kwa Project
Manage vehicles	Anaweza kusaidia kusajili au kurekebisha vehicle za customers.	Create, read, update, delete vehicles.
Manage parking slots	Anaongeza au kubadilisha parking bay kama Standard, VIP, Disabled.	Create, read, update, delete slots.
Check-in/check-out	Anaweza kuingiza au kutoa vehicle yoyote kwenye parking.	Admin can operate any vehicle.
Payment handling	Anaweza kupokea malipo ya parking.	Cash, card, mobile money payment.
Reports	Anaangalia mapato na matumizi ya parking.	Revenue report and occupancy report.

Admin role in code: ADMIN. Inapatikana kwenye Admin.java.

3. Customer Anafanya Nini?

Kwa real life, Customer ni mwanafunzi, staff, au mtu mwenye vehicle anayetumia parking.

Customer Role	Maelezo Ya Real Life	Kwa Project
Register own vehicle	Customer anaingiza plate number ya gari lake.	Create/read/update/delete own vehicle if allowed.
Use parking	Customer ana-park vehicle yake tu.	Customer can operate only own vehicle.
Check-in/check-out own vehicle	Anaingiza vehicle parking na kuitoa anapomaliza.	Parking session for own vehicle only.
Pay parking fee	Analipa baada ya check-out.	Cash, Card, or Mobile Money.
View own history	Anaona matumizi yake ya parking.	Customer sees own sessions only.

Customer role in code: `CUSTOMER`. Inapatikana kwenye `Customer.java`.

4. Application Stack

Layer	Technology	Kazi Yake
Frontend	React + TypeScript + Vite	UI ambayo user anaona: login, vehicles, lot map, check-in, check-out, reports.
Backend	Spring Boot + Java	Business logic, REST APIs, validation, security, services.
Database	PostgreSQL	Inahifadhi users, vehicles, slots, sessions, tickets, payments.
ORM	Spring Data JPA + Hibernate	Inaunganisha Java classes na database tables.

5. Project Modules Zipo 5 Au Zaidi?

Ndiyo. Kwa functional modules, project ina zaidi ya modules 5. Modules hapa maana yake ni sehemu kubwa za system zenye kazi maalum.

#	Module	Kazi Yake
1	Authentication/User Module	Login, logout, current user, admin/customer roles.
2	Vehicle Module	Create, read, update, delete vehicles.
3	Parking Slot Module	Create, read, update, delete parking slots.
4	Parking Session Module	Check-in, check-out, active/closed sessions.
5	Payment Module	Cash, card, mobile money payments.
6	Ticket Module	Ticket generation when vehicle checks in.
7	Report Module	Revenue and occupancy reports for admin.
8	Frontend UI Module	React pages and controls for users.

6. Main Model/Domain Files

The project has **25 domain files**: 17 entity model classes and 8 enums/helper domain types.

Important entities: User, Admin, Customer, Vehicle, Car, Truck, Motorcycle, ParkingSlot, StandardSlot, VipSlot, DisabledSlot, ParkingSession, Ticket, Payment, CashPayment, CardPayment, MobileMoneyPayment.

Enums/helper types: VehicleType, SlotType, SlotSize, SessionStatus, PaymentMethod, PaymentStatus, MobileMoneyProvider, PaymentResult.

7. Java Basics Used In This Project

Java Basic	Simple Meaning	Example In Project
Class	Blueprint ya object.	<code>Car, Customer, ParkingSession</code>
Object	Real item created from class.	<code>new Car("T123ABC", owner, 4)</code>
Method	Function inside a class.	<code>calculateFee(), checkIn(), process()</code>
Constructor	Special method used when creating object.	<code>public Car(String plateNumber, Customer owner, int doors)</code>
Enum	Fixed list of values.	<code>VehicleType, PaymentMethod, SlotSize</code>
List	Collection ya items nyingi.	<code>List<Vehicle>, List<ParkingSession></code>
Annotation	Metadata for Spring/JPA.	<code>@Entity, @Service, @RestController</code>

8. OOP Principles Used In This Project

OOP Principle	Meaning Rahisi	Project Example
Encapsulation	Data inalindwa. Unabadilisha through methods, si direct access.	<code>Vehicle.setPlateNumber()</code> validates plate format before saving.
Inheritance	Class moja inarithi from parent class.	<code>Car extends Vehicle, Admin extends User.</code>
Abstraction	Parent class defines idea, child classes complete details.	<code>Vehicle</code> has abstract <code>hourlyRate()</code> .
Polymorphism	Same method name, different behavior depending on object.	<code>hourlyRate()</code> returns different rate for <code>Car</code> , <code>Truck</code> , <code>Motorcycle</code> .
Override	Child class replaces parent abstract method implementation.	<code>@Override public String roleName()</code> in <code>Admin</code> and <code>Customer</code> .

9. Simple OOP Code Examples From This Project

A. Abstraction and Inheritance: User, Admin, Customer

```
public abstract class User {
    public abstract String roleName();
    public abstract boolean canOperate(Vehicle vehicle);
}

public class Admin extends User {
    @Override
    public String roleName() {
        return "ADMIN";
    }

    @Override
    public boolean canOperate(Vehicle vehicle) {
        return vehicle != null;
    }
}

public class Customer extends User {
    @Override
    public String roleName() {
        return "CUSTOMER";
    }

    @Override
    public boolean canOperate(Vehicle vehicle) {
        return vehicle != null && this.equals(vehicle.getOwner());
    }
}
```

Meaning: Admin can operate any vehicle. Customer can operate only own vehicle.

B. Encapsulation: Getter and Setter In Vehicle

```
public void setPlateNumber(String rawPlate) {
    if (rawPlate == null) {
        throw AppException.badRequest("Invalid plate number.");
    }
    String normalized = rawPlate.trim().toUpperCase(Locale.ROOT);
    if (!PLATE_PATTERN.matcher(normalized).matches()) {
        throw AppException.badRequest("Expected format T123ABC.");
    }
    this.plateNumber = normalized;
}

public String getPlateNumber() {
    return plateNumber;
}
```

Meaning: Plate number haiwekwi directly. Setter kwanza inacheck kama format ni sahihi.

C. Polymorphism: Vehicle Rates

```
public abstract class Vehicle {
    public abstract BigDecimal hourlyRate();
    public abstract SlotSize requiredSize();
}

public class Car extends Vehicle {
    @Override
    public BigDecimal hourlyRate() {
        return new BigDecimal("2000");
    }

    @Override
    public SlotSize requiredSize() {
        return SlotSize.MEDIUM;
    }
}

public class Truck extends Vehicle {
    @Override
    public BigDecimal hourlyRate() {
        return new BigDecimal("5000");
    }

    @Override
    public SlotSize requiredSize() {
        return SlotSize.LARGE;
    }
}
```

Meaning: Same method `hourlyRate()` works differently depending on vehicle type.

D. Getter and Setter: Car Details

```
public int getDoors() {
    return doors;
}

public void setDoors(int doors) {
    if (doors < 2 || doors > 6) {
        throw new IllegalArgumentException("Car doors must be between 2 and 6.");
    }
    this.doors = doors;
}
```

Meaning: Getter reads value. Setter changes value with validation.

E. Polymorphism In Payment

```
public abstract class Payment {
    public abstract PaymentResult process();
}

public class CashPayment extends Payment {
    @Override
    public PaymentResult process() {
        if (amountTendered.compareTo(getAmount()) < 0) {
            return PaymentResult.failed("Cash is not enough.");
        }
        return PaymentResult.success("CASH-" + getSession().getId(), "Paid.");
    }
}
```

Meaning: Cash, Card, and Mobile Money all use process(), but each one processes payment differently.

10. CRUD In This Project

CRUD means Create, Read, Update, Delete. Hii project sasa ina CRUD inayoonekana kwenye UI kwa Vehicles na Parking Slots.

Module	Create	Read	Update	Delete
Vehicles	Register vehicle	View all/my vehicles	Edit button in Vehicles page	Delete button in Vehicles page
Parking Slots	Create slot form on Lot map for Admin	View bays on Lot map	Edit button on each slot for Admin	Delete button on each slot for Admin

Vehicle CRUD API Example

```
@RestController
@RequestMapping("/api/vehicles")
public class VehicleController {
    @GetMapping
    public List<VehicleResponse> list() { ... }

    @PostMapping
    public VehicleResponse register(@RequestBody RegisterVehicleRequest request) { ... }

    @PutMapping("/{id}")
    public VehicleResponse update(@PathVariable Long id,
                                  @RequestBody UpdateVehicleRequest request) { ... }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) { ... }
}
```

Slot CRUD API Example

```
@RestController
@RequestMapping("/api/slots")
public class SlotController {
    @GetMapping
    public List<SlotResponse> list() { ... }

    @PostMapping
    public SlotResponse create(@RequestBody CreateSlotRequest request) { ... }

    @PatchMapping("/{code}")
    public SlotResponse update(@PathVariable String code,
                               @RequestBody UpdateSlotRequest request) { ... }

    @DeleteMapping("/{code}")
    public void delete(@PathVariable String code) { ... }
}
```

11. Main Flow Ya Application

1. **Login:** User enters username and password.

2. **Vehicle:** User registers or manages vehicle.
3. **Slot:** System checks available parking slot.
4. **Check-in:** Vehicle is assigned to a slot and ticket is issued.
5. **Check-out:** System calculates fee based on time, vehicle rate, and slot multiplier.
6. **Payment:** User pays by cash, card, or mobile money.
7. **Reports:** Admin views revenue and occupancy.

12. Fee Calculation Logic

```
public BigDecimal calculateFee() {  
    return vehicle.hourlyRate()  
        .multiply(slot.rateMultiplier())  
        .multiply(BigDecimal.valueOf(billableHours()))  
        .setScale(2, RoundingMode.HALF_UP);  
}
```

Meaning rahisi: Parking fee = vehicle hourly rate x slot multiplier x hours.

- Car: TSh 2,000 per hour.
- Truck: TSh 5,000 per hour.
- Motorcycle: TSh 1,000 per hour.
- VIP slot multiplier: 1.5.
- Partial hours are rounded up.

13. Frontend React Example

```
async function handleDelete(vehicle: VehicleResponse) {  
    if (!window.confirm(`Delete vehicle ${vehicle.plateNumber}?`)) {  
        return;  
    }  
    await api.delete<void>(`/api/vehicles/${vehicle.id}`);  
    load();  
}
```

Meaning: React UI asks for confirmation, calls backend DELETE API, then reloads the vehicle list.

14. Backend Spring Boot Example

```
@Service  
@Transactional  
public class VehicleService {  
    public void delete(User actor, Long id) {  
        Vehicle vehicle = requireById(id);  
        requireCanManage(actor, vehicle);  
  
        if (sessionRepository.existsByVehicle(vehicle)) {  
            throw AppException.conflict("Cannot delete vehicle with parking history.");  
        }  
  
        vehicleRepository.delete(vehicle);  
    }  
}
```

Meaning: Backend checks permission and safety before deleting a vehicle.

15. Database PostgreSQL

Database stores application data. Spring Data JPA repositories help backend read/write data without writing raw SQL for every operation.

Data	Stored As
Users	Admin and Customer records.
Vehicles	Car, Truck, Motorcycle records.
Slots	Standard, VIP, Disabled parking bays.
Sessions	Check-in/check-out history.
Payments	Cash, Card, Mobile Money transactions.
Tickets	Ticket code issued during check-in.

16. Short Summary

Hii ni parking management application yenye Admin na Customer. Admin anasimamia parking nzima, while Customer anasimamia vehicle zake mwenyewe. Project ina frontend ya React, backend ya Spring Boot, database ya PostgreSQL, ina OOP principles, na ina modules zaidi ya 5. CRUD inaonekana kwenye UI kwa Vehicles na Parking Slots.

Generated from the local project source code.